Master IASD

DATALAB – Project A1

Using Personality Diagnosis for Collaborative Filtering

Collaborative Filtering for Movie Rating Prediction

Blocus: Group 15

Cenzo Poirier De Carvalho
Théodore Labrande
Axel Belloir

Academic year 2026–2027

1 Introduction

Recommender systems predict users' preferences from historical interactions. Collaborative filtering (CF) exploits patterns in user–item ratings, without a detailed model of each user's preferences or each item's content. This project predicts the missing ratings of a partially observed matrix of 610 users and 4,980 movies, a sparse matrix completion problem. We implement matrix factorization by gradient descent, investigate alternative approaches and compare them with a baseline, considering accuracy, hyperparameters, regularization, data splitting and computational cost. Movie genre metadata is also available as item-content information. We ask how well matrix factorization predicts unseen ratings, how it compares with memory-based methods, whether genre information improves predictions, and how models should be evaluated for a fair comparison.

2 Problem Formulation and Experimental Protocol

2.1 Rating prediction

Let $R \in \mathbb{R}^{U \times I}$ be the user–item rating matrix ($U = 610$, $I = 4980$), r_{ui} the rating of user u for movie i, and $\Omega = \{(u, i) : r_{ui} \text{ is observed}\}$. Parameters are learned on training observations and predictions are evaluated on held-out ratings with the root mean squared error

$$\text{RMSE} = \sqrt{\frac{1}{|\Omega_{\text{eval}}|} \sum_{(u,i) \in \Omega_{\text{eval}}} (r_{ui} - \hat{r}_{ui})^2},$$

where Ω_{eval} is the set of held-out ratings and $\hat{r}_{ui}$ the prediction. Lower is better, and larger errors are penalized more strongly.

2.2 Data splitting and comparison protocol

The provided train and test matrices separate model selection from final evaluation: hyperparameters are compared on a validation subset sampled from the training observations, and the test set is reserved for the final comparison. Methods compared directly use the same evaluation entries, and a simple baseline is the reference against which added complexity is judged.

The platform receives train *and* test and is scored on a hidden set, so its input is denser than train alone (median 46 ratings per user instead of 24), which matters for neighbourhood methods. We therefore use two settings. **V** (validation, used for every choice): input is train + test minus 30% of the train ratings (9,479 hidden), seeds 0–2, so its input has 85% of the ratings the platform receives; no test rating is ever a target. **T** (local test): train as input, test as target. We report RMSE, exact-rating accuracy (prediction rounded to the nearest half-star equals the true rating; our reading of the platform's metric) and time; validation gives the mean ± standard deviation over three seeds, on identical hidden sets, so comparisons are paired. Caveat: the structural ideas were first explored on similar splits, so test is not perfectly untouched, and our search was coordinate-wise, not global.

The implementation also uses the provided fixed movie metadata file `namesngenre2.npy`. Its rows contain movie titles and genre labels, while the rating matrix columns correspond to movie IDs. Metadata must therefore be aligned with the rating columns by the provided ordering, not by title: different movie IDs can share the same title.

3 Matrix Factorization

3.1 Model

Our matrix factorization is a biased one: a global mean and user and movie biases sit under the latent factors. Each user u has a latent vector $q_u \in \mathbb{R}^f$ and each movie i a latent vector $p_i \in \mathbb{R}^f$, where f is the number of latent factors. The predicted rating is

$$\hat{r}_{ui} = \mu + b_u + b_i + q_u^\top p_i,$$

where μ is the global mean rating and b_u, b_i are user and movie biases; the dot product models interactions that the biases cannot explain.

3.2 Optimization and regularization

The parameters minimize a regularized squared error over the observed training ratings:

$$\mathcal{L} = \sum_{(u,i)\in\Omega_{\text{train}}} (r_{ui} - \hat{r}_{ui})^2 + \lambda\Big(\sum_u \|q_u\|^2 + \sum_i \|p_i\|^2 + \sum_u b_u^2 + \sum_i b_i^2\Big).$$

The coefficient λ penalizes large parameters and reduces overfitting. Gradient descent updates the parameters from the prediction errors, with a learning rate (step size) and a number of epochs (passes). The main hyperparameters are the number of factors, learning rate, regularization, epochs and random seed; more factors capture richer preference patterns but increase cost and the risk of overfitting.

3.3 Implementation

The implementation is `mf.py`, function `matrix_factorization_gd`: global, user and item biases with latent factors, trained by gradient updates on observed ratings. An unobserved rating does not mean that the user disliked the movie, so the objective is computed only on observed training entries rather than treating missing entries as zeros.

4 Our Method of Interest: Personality Diagnosis

4.1 Principle

Personality Diagnosis (PD), introduced by David Pennock [1] and other researchers at Microsoft and NEC, is a memory-based method: instead of learning latent representations, it predicts a user's rating from other users with similar observed preferences. It calls itself a hybrid method, but could essentially be seen as an extreme type of k-NN. For a target user u and a movie i, the candidates are the users v who rated i; the ratings of u on other movies tell which candidates have a similar profile. Under a Gaussian noise assumption, the likelihood contribution of a shared rating is

$$P(r_{uj} \mid r_{vj}) \propto \exp\left(-\frac{(r_{uj} - r_{vj})^2}{2\sigma^2}\right),$$

where σ controls how strongly rating differences matter: a small σ gives much more weight to close matches, a large one gives a similar weight to all users. The contributions over the movies rated by both users are combined into the plausibility of each candidate profile. New inferred ratings come from taking the mode of the posterior over the candidates' ratings for the target movie.

4.2 From the paper's PD to our method

PD weights user b for predicting user a by $w_{ab} = \prod_{k\in J_{ab}} \exp(-(r_{ak} - r_{bk})^2/2\sigma^2)$ over the items J_{ab} both rated. The paper predicts the mode of the weighted mixture; since RMSE is minimised by the mean, we predict the expectation $\hat{r}_{ak} = \sum_b w_{ab} r_{bk} / \sum_b w_{ab}$. As published, with σ chosen on validation, it gives 0.970 on validation and 1.009 on test: *worse than the bias baseline* (0.859, 0.874). Raw ratings mix tastes with habits; sparsity turns the product into noise (two users with a movie in common share a median of 4; only 25% of user pairs and 1% of movie pairs share 8 or more), so a neighbour sharing 2 movies can outweigh one sharing 20; and nothing makes the prediction fall back to a neutral value when no neighbour is informative.

We changed PD step by step and kept what lowered the validation RMSE. Let $b_{ui} = \mu + b_u + b_i$ be a bias baseline (regularised offsets, $\lambda_I = 3$, $\lambda_U = 5$), $z_{ui} = r_{ui} - b_{ui}$ the residuals and m the mask of known ratings. For two rows a, b (movies or users) with c_{ab} common columns, one *PD stage* is

$$d_{ab} = \frac{1}{c_{ab}} \sum_k m_{ak} m_{bk} (z_{ak} - z_{bk})^2, \quad w_{ab} = e^{-d_{ab}/2\sigma^2} \frac{c_{ab}}{c_{ab} + \lambda} \ \ (c_{ab} \geq c_{\min}), \quad \hat{z}_{ak} = \frac{\sum_b w_{ab} m_{bk} z_{bk}}{\kappa + \sum_b w_{ab} m_{bk}}, \quad (1)$$

optionally keeping the K largest weights per row. Stages are chained: each works on the residuals the previous ones left, and the prediction is $b + \sum_s \hat{z}^{(s)}$, clipped to $[0.5, 5]$. Validation RMSE after each step:

(1) *Residuals instead of raw ratings*, paper weights: 0.970 → 0.877, still above the baseline.
(2) *Mean gap, confidence* $c/(c+\lambda)$, $c \geq c_{\min}$ *and shrinkage* κ toward the baseline: → 0.853, below the baseline. Known ideas [2, 4], tested here on our data.

(3) *Movies as neighbours* [3]: → 0.849.
(4) *Three chained stages* (movies, users, movies), like boosting: → 0.843; no gain on test (0.868 → 0.868), whose input is half as dense.
(5) *Best neighbours:* stage 1 keeps the $K = 100$ best neighbours of each movie, re-tuned: → 0.837. Final stages $(\sigma, \kappa, \lambda, c_{\min})$: movies $(1.5, 0.25, 640, 8)$ with $K = 100$; users $(0.5, 0.5, 80, 5)$; movies $(0.7, 0.25, 80, 3)$.

5 Genre-Based and Hybrid Recommendation

5.1 Genre representation

The metadata gives each movie a pipe-separated genre list and a title ending with its year. Movie j is encoded by $f_j \in \{0,1\}^{27}$: 20 multi-hot genre indicators and 7 era indicators (before 1950, 1950–69, 1970s, 1980s, 1990–94, 1995–99, 2000 onward).

5.2 Combining genre information and PD

Genres enter as a sixth step after the three PD stages. With $\tilde{z}_{uj}$ the residuals they leave, each user gets a ridge regression on the movies he or she rated, $w_u = \left(\sum_j m_{uj} f_j f_j^\top + \lambda_m I\right)^{-1} \sum_j m_{uj} \tilde{z}_{uj} f_j$, and the prediction is raised by $f_j^\top w_u$ ($\lambda_m = 20$): 27 parameters per user, fitted on his or her own ratings and shrunk to zero when there are few. Validation went from 0.837 to 0.818 and test from 0.865 to 0.845, a gain on every seed. Unlike the chained stages, the gain holds on test. Alternatives on validation: the user's mean residual per genre 0.8245; ridge on genres alone 0.8230 (era adds −0.0045); genre-overlap neighbours on top −0.0003; moving the step within the chain changes validation by under 0.0015.

5.3 Implementation details

The code is `pd_ameliore.py`, evaluated by `evaluate_pd_ameliore.py`. We checked the alignment of the metadata on the three most-rated columns (Forrest Gump, Shawshank Redemption, Pulp Fiction). The metadata file is not in the repository: `generate.py` uses it if it finds it next to the ratings or the script, and otherwise stops at step 5, with predictions identical to the version scored 0.833. *Libraries:* NumPy only in the submitted code (`mf.py`, `pd_ameliore.py`); SciPy for the unsubmitted Bayesian factorization, matplotlib for the figures; no external recommender code: the PD of the paper and the ideas of Section 4.2 are re-implemented.

6 Experiments and Results

6.1 Experimental setup

Protocol and seeds are those of Section 2 (validation, test). Times are one fit on the train file on a 2-core machine. The matrix factorization is our gradient-descent implementation `mf.py` (Section 3), run with its defaults (32 factors, learning rate 0.005, $\lambda = 0.02$, 30 epochs) and, separately, tuned on validation (below). Scripts: `run_experiments.py` (tables), `tune_mf.py`, `make_report_figures.py`; seeds are fixed and a rerun gives the same numbers.

6.2 Hyperparameter selection

Figure 1 moves one hyperparameter at a time around the final PD stages (validation, without step 6). The optimum is flat: our values are within 0.0006 of the best of every curve, and 41 of the 47 alternatives cost less than 0.002. The costly directions are a small σ or a weak confidence at stage 1 ($\sigma = 0.5$: +0.0036; $\lambda = 40$: +0.0039), a stage-2 σ too small or too large (+0.0031, +0.0029) and a strict $c_{\min}$ at stage 3 (+0.0044).

Regularization: the baseline alone is sensitive to its ridge terms (RMSE 0.8589 to 0.8818 over $(1, 1)$ to $(20, 20)$), the PD stages much less (0.8366 to 0.8412); the shrinkage κ, the confidence λ and the genre ridge λ_m play the same role in the stages. The number of stages matters in order (0 to 3 stages: 0.859, 0.850, 0.843, 0.837), not their order (users first: 0.837). *Splits:* the standard deviation of validation across seeds is 0.0031, so differences below about 0.001 (e.g. $K = 50$ against 100) are not resolved.

Matrix factorization. Coordinate search of `mf.py` on validation: 23 settings, a sweep around the defaults (Table 1), then neighbours of the best combination. The choice (8 factors, rate 0.005, $\lambda = 0.1$, 60 epochs) gives 0.859 on validation and 0.877 on test (defaults: 0.872, 0.885). More capacity, faster or longer

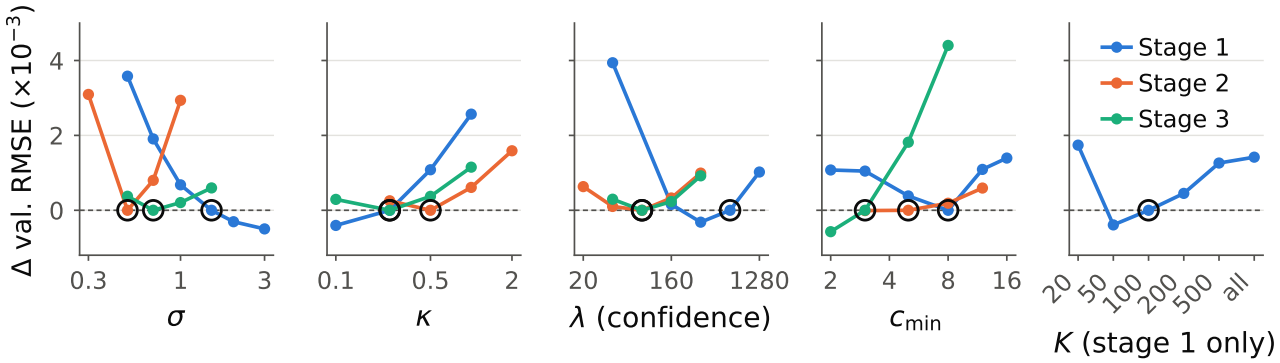


Figure 1: Change of validation RMSE ($\times 10^{-3}$) when one hyperparameter moves; circles mark the chosen values.

training and weaker regularization hurt; a stronger λ makes longer training pay (60 epochs: 0.904 at $\lambda = 0.02$, 0.859 at $\lambda = 0.1$). Tuned, it only matches the bias baseline on validation (0.859) and stays 0.003 above it on test (0.874). Validation chose among 23 settings, so it is slightly optimistic; test played no part.

Factors	8: 0.866, 16: 0.868, **32: 0.872**, 64: 0.873, 128: 0.879
Learning rate	0.002: 0.875, **0.005: 0.872**, 0.01: 0.903, 0.02: 0.934
λ	0.005: 0.886, **0.02: 0.872**, 0.05: 0.864, 0.1: 0.863, 0.2: 0.866
Epochs	15: 0.872, **30: 0.872**, 60: 0.904, 100: 0.928

Table 1: Validation RMSE of `mf.py`, one hyperparameter changed at a time from the defaults in bold (value: RMSE).

6.3 Predictive performance

Method	Val (RMSE)	Test (RMSE)	Test acc. (%)	Test time (s)
Matrix factorization, defaults	0.872 ± 0.005	0.885	24.3	10.3
Matrix factorization, tuned	0.859 ± 0.005	0.877	24.4	22.8
Bias baseline	0.859 ± 0.005	0.874	24.4	0.3
PD of the paper	0.970 ± 0.005	1.009	21.4	0.5
(1) PD on residuals, paper weights	0.877 ± 0.006	0.913	24.0	0.3
(2) + mean gap, confidence, shrinkage	0.853 ± 0.005	0.872	24.6	0.3
(3) neighbours = movies	0.849 ± 0.004	0.868	24.7	1.4
(4) + 3 chained stages	0.843 ± 0.003	0.868	24.8	2.7
(5) + best 100 neighbours (PD scored 0.833)	0.837 ± 0.004	0.865	24.9	2.8
(6) + genres and era (final)	**0.818 ± 0.002**	**0.845**	**25.3**	**3.3**

Table 2: Results of each step. Val: mean ± sd over three seeds; time: one fit on the train file.

The final method gains 0.041 on validation over the bias baseline, 0.041 over the tuned factorization (0.032 on test) and 0.053 over the default one (0.041 on test). For the PD stages the gain over the baseline grows with the data, so cold-start users and movies gain least: 0.019 for users with fewer than 25 ratings against 0.026 for users with 200 or more, and 0.011 for movies with at most 2 ratings against 0.034 for movies with 30 or more. Exact-rating accuracy hardly moves (24.4% to 25.3% on test): we minimise squared error, not accuracy.

Platform. Without step (6) the PD scored **0.833** on the platform, against 0.867 for our factorization with default settings (the tuned one was not submitted). Validation tracks it: 0.837 and 0.872, about 0.004 and 0.005 above, with the same gap (0.035 on validation, 0.034 on the platform). With step (6), validation gives 0.818, hence about 0.81 on the platform if the metadata file is available there, else 0.833.

6.4 Computational cost

The PD has no training loop: `generate.py` takes 3.6 s and about 705 MB, against 12.4 s (default factorization) and 27.3 s (tuned); the platform allows 1,200 s. The two movie stages dominate ($4{,}980 \times 4{,}980$ single-precision matrices); step (6) adds 0.5 s. Not submitted: mixing the PD with a Bayesian probabilistic factorization [5], or stacking both, reach 0.867 and 0.860 on test (PD stages: 0.865) at 31 and 57 times the cost.

7 Discussion

7.1 What worked and what failed

The gap between the paper's PD and ours comes from what surrounds the Gaussian weight, not from its form: removing user and movie effects, weighting by the amount of evidence, shrinking toward the baseline. Chaining stages is cheap and works like boosting. Genres and era give a gain (0.018 on validation) larger than steps (3)–(5) together (0.017). No meaningful gain (validation RMSE changes from -0.0005 to $+0.0094$ for the first six): sharpened weights w^p, a gain on the predicted residual, one joint vote over movie and user neighbours, a regression-slope variant, re-fitting biases between stages, a fourth or fifth stage; SVD++ and ALS on ratings only. A random walk on a user–movie–genre graph (He et al., AAAI 2011) was tried only with the misaligned metadata file.

7.2 Impact of hyperparameters and regularization

A small σ or a weak confidence at stage 1 is costly: a sharp weight on few common ratings is unreliable, which the confidence term and κ guard against. Stage 1 asks for much common evidence ($\lambda = 640$, $c_{\min} = 8$), stage 3 little ($c_{\min} = 3$); validation selects this split, we have not tested why.

7.3 Limitations and lessons learned

Steps (1)–(3) were not re-tuned on validation; the factorization got a short coordinate search, not a global one; one split scheme only (30% of the train ratings hidden); the era bins are one hand-chosen partition; the platform gives a single score and may not provide the metadata file; why movie neighbours win is not established; our accuracy is a reading of the metric. Our hyperparameters suit the platform's density, so the stage gain is smaller on test. A misaligned metadata file hid the gain of genres: check alignment first.

8 Conclusion

A PD on raw ratings is worse than a bias baseline at 2% density; working on residuals, weighting by the amount of evidence with shrinkage toward the baseline, and chaining movie and user stages make it beat both the baseline and our gradient-descent factorization (0.833 against 0.867 on the platform, default settings; tuned, the factorization only reaches the baseline). Genres and era add a further gain through a per-user ridge regression. Next: re-tune steps (1)–(3) on validation and combine the PD with a metadata-aware factorization.

References

[1] D. M. Pennock, E. Horvitz, S. Lawrence, and C. L. Giles. Collaborative Filtering by Personality Diagnosis: A Hybrid Memory- and Model-Based Approach. In *Proceedings of the Sixteenth Conference on Uncertainty in Artificial Intelligence (UAI)*, 2000. https://arxiv.org/abs/1301.3885

[2] J. L. Herlocker et al. An algorithmic framework for performing collaborative filtering. *SIGIR*, 1999.

[3] B. Sarwar et al. Item-based collaborative filtering recommendation algorithms. *WWW*, 2001.

[4] Y. Koren. Factor in the neighbors: scalable and accurate collaborative filtering. *TKDD*, 2010.

[5] R. Salakhutdinov, A. Mnih. Bayesian probabilistic matrix factorization using MCMC. *ICML*, 2008.